Autonomous Emergency Braking System (AEBS)

System Specification and Formal Abstraction

Heba Al Kayed

Abstract

This document formally specifies the dynamics of the AEBS case study. It is divided into two primary sections. **Part I** defines the concrete simulation environment, modelled as a Discrete-Time Stochastic Hybrid System (dt-SHS), detailing the physical plant, sensor noise, and controller logic. **Part II** details the formal abstraction pipeline, which discretizes the concrete dynamics into an Interval Markov Decision Process (IMDP) for formal verification using the PRISM model checker. The abstraction methodology explicitly adapts finite approximate abstraction techniques for dt-SHS.

Part I: Concrete System Dynamics

1 System Architecture

The system is modelled as a closed-loop simulation consisting of four distinct components:

- **Vehicle Plant ($\mathcal{P}$):** A discrete-time physical model of the ego vehicle.
- **Traffic Environment ($\mathcal{E}$):** Manages the dynamics of the lead vehicle and scenario configuration.
- **Perception System ($\mathcal{S}$):** Acts as a stochastic observer function $\mathbf{y} = f(\mathbf{x})$.
- **Controller ($\mathcal{C}$):** A deterministic reactive policy mapping observations to control inputs.

2 Vehicle Plant Dynamics

The vehicle dynamics are implemented in the `VehiclePlant` class. The system state evolves in discrete time steps of $\Delta t = 0.1s$.

2.1 State Space Definition

The state vector is defined as $\mathbf{x}_k = [x_k, v_k, a_k]^T$, where:

- x_k: Position (or Gap) at step k.
- v_k: Velocity (or Closing Speed) at step k.
- a_k: Longitudinal Acceleration at step k.

The system supports two coordinate references:

- **World Frame:** x denotes absolute position on the track (increasing).
- **Relative Frame:** x denotes the **Gap** to the obstacle (decreasing). This frame is utilized for formal abstraction.

2.2 Discrete-Time Update Law

The physics engine implements a **Semi-Implicit Euler** update scheme. Unlike basic Euler integration, this method uses the updated acceleration to compute velocity, and the *updated* velocity to compute position.

This distinction is crucial for simulation fidelity. Semi-Implicit Euler is symplectic, meaning it better preserves the physical structure of the system's phase space. For mechanical systems, it tends to dampen artificial energy growth, behaving more like the true continuous flow than standard explicit methods.

The state update equations for the transition $t_k \to t_{k+1}$ are executed sequentially as follows:

1. Actuator Lag (Acceleration Update)

$$a_{k+1} = a_k + \alpha \cdot (u_k - a_k) \tag{1}$$

Where:

- u_k: The numeric control input requested by the controller at step k. This is a continuous value representing the target longitudinal acceleration in m/s^2 (e.g., $u_k = -9.8$ for emergency braking).
- α: The actuator lag coefficient ($\alpha = 0.5$), representing the physical delay in the braking system's response.

Explanation: Real-world vehicles cannot change acceleration instantaneously due to hydraulic and mechanical delays. Equation 1 acts as a discrete-time first-order low-pass filter to model this. If a vehicle is coasting ($a_k = 0.0\, m/s^2$) and the controller suddenly commands an emergency stop ($u_k = -9.8\, m/s^2$), the actual acceleration in the first $0.1s$ time step becomes:

$$a_{k+1} = 0.0 + 0.5 \cdot (-9.8 - 0.0) = -4.9\, m/s^2$$

Over subsequent steps, the acceleration asymptotically approaches the commanded $-9.8\, m/s^2$, accurately mirroring the physical accumulation of hydraulic brake pressure over time.

2. Kinematic Velocity Update

$$v_{k+1} = \max(0, v_k + a_{k+1} \cdot \Delta t) \tag{2}$$

Where Δt is the time step duration ($0.1s$).

Explanation: This computes the new closing speed based on standard kinematics ($v = v_0 + at$), but distinctly utilizes the *newly calculated* acceleration (a_{k+1}) from Equation 1. The outer $\max(0, \dots)$ function acts as a physical boundary constraint: when a vehicle brakes to a complete stop, it does not begin accelerating backwards. This mathematically guarantees the velocity cannot drop below zero.

3. Relative Position Update

$$x_{k+1} = \begin{cases} x_k + v_{k+1} \cdot \Delta t & \text{if world\_frame} \\ x_k - v_{k+1} \cdot \Delta t & \text{if relative\_frame} \end{cases} \tag{3}$$

Explanation: Finally, the position is updated using the *newly calculated* velocity (v_{k+1}), completing the symplectic chain. For the formal verification abstraction, the model operates in the `relative_frame`. In this frame, x represents the spatial **gap** between the ego vehicle and the obstacle. Therefore, a positive closing speed (v_{k+1}) actively *reduces* the gap distance over the time step Δt, resulting in a subtraction operation.

```
# 1. Update Acceleration (First-Order Lag)
# Maps to Eq (1): a_new = a + alpha * (u - a)
self.actual_acceleration = self.actual_acceleration + self.alpha * (action_acceleration
    - self.actual_acceleration)

# 2. Update Velocity
# Maps to Eq (2): v_new = v + a_new * dt
self.actual_velocity += self.actual_acceleration * self.dt

# Constraints: No negative velocity
if self.actual_velocity < 0.0:
    self.actual_velocity = 0.0
```

```
        self.actual_acceleration = 0.0

    # 3. Update Position
    # Maps to Eq (3): x_new = x + v_new * dt
    if self.coordinate_system == 'world_frame':
        self.x += self.actual_velocity * self.dt
    else:
        # Relative Frame: x is GAP. Positive closing speed REDUCES the gap.
        self.x -= self.actual_velocity * self.dt
```

Listing 1: Implementation of Semi-Implicit Euler Step

3 Perception and Sensor Noise Model

The `PerceptionSystem` implements the observation function. It introduces stochasticity via two mechanisms: aleatoric measurement noise (Gaussian jitter) and probabilistic detection failure (Bernoulli dropouts).

3.1 Observation Function

The observed state $\mathbf{y}_k = [\tilde{g}, \tilde{v}]$ is derived from the ground truth $\mathbf{x}_k$:

$$\mathbf{y}_k = \begin{cases} \emptyset(\text{False}) & \text{with probability } P_{FN} \\ \mathbf{x}_k + \mathbf{w}_k & \text{otherwise} \end{cases} \tag{4}$$

Where:

- P_{FN}: The False Negative rate (probability the sensor misses the car entirely).
- $\mathbf{w}_k$: The noise vector drawn from independent Gaussian distributions.

$$\mathbf{w}_k \sim \mathcal{N}(0, \mathbf{\Sigma}), \quad \mathbf{\Sigma} = \begin{bmatrix} \sigma_{pos}^2 & 0 \\ 0 & \sigma_{vel}^2 \end{bmatrix}$$

3.2 Sensor Noise Justification

The variance of the Gaussian noise $\mathbf{\Sigma}$ is configured to reflect the real-world operating characteristics of standard automotive radar and camera systems.

For a discrete time step of $\Delta t = 0.1s$, the measurement noise must capture the sensor's instantaneous uncertainty. Commercial automotive radars typically exhibit a range accuracy error of approximately $\pm 0.1m$ to $\pm 0.5m$, and a relative velocity accuracy of $\pm 0.1m/s$.

Therefore, the baseline standard deviations are defined as $\sigma_{pos} = 0.5$ and $\sigma_{vel} = 0.1$. These values represent a realistic degree of aleatoric uncertainty in the perception layer, which the formal abstraction must bound and account for during verification.

```
# 1. Detection Logic (Classification DNN)
# Maps to P_FN logic
if np.random.rand() < self.fn_rate:
    return False, real_gap, real_v_rel

# 2. Estimation Logic (Regression DNN)
# Maps to w_k ~ N(0, Sigma)
gap_noise = np.random.normal(0, self.pos_noise)
vel_noise = np.random.normal(0, self.vel_noise)

obs_gap = max(0.0, real_gap + gap_noise)
obs_v_rel = real_v_rel + vel_noise
```

Listing 2: Sensor Noise Injection

4 Controller Logic

The controller implements a reactive decision logic. It does not access the ground truth state $\mathbf{x}_k$; instead, it relies entirely on the noisy observation $\mathbf{y}_k$ and its own internal belief of its velocity state.

To support comprehensive benchmarking, the system implements two distinct parameter sets representing different design philosophies.

Table 1: Controller Benchmark Parameters (Comparison)

Parameter	Symbol	Industry Mode	Safe Mode
Time Thresholds			
Warning TTC	TTC_{warn}	2.6s	6.0s
Braking TTC	TTC_{brake}	1.6s	5.0s
Emergency TTC	TTC_{crit}	1.0s	4.0s
Distance Thresholds			
Warning Dist	d_{warn}	10.0m	15.0m
Braking Dist	d_{brake}	6.0m	10.0m
Emergency Dist	d_{crit}	2.0m	5.0m
Actuation			
Brake Force	u_{brake}	-4.0 m/s^2	-4.0 m/s^2
Emergency Force	u_{crit}	-9.8 m/s^2	-8.0 m/s^2

4.1 Decision Logic Implementation

The logic branches based on observed closing velocity ($v_{rel} < 5.0m/s$) to distinguish between low-speed approach (using distance thresholds) and high-speed collision avoidance (using TTC thresholds).

```
# 1. Update Internal Belief (State Estimation)
# The controller tracks its own estimated velocity, separate from ground truth
self.est_velocity = max(0.0, prediction + noise)

# 2. Decision Logic (Using Observed Gap/Velocity)
if obs_v_rel < 5.0:
    if obs_gap < self.dist_emergency: req_acc, self.state = self.acc_emergency, '
    emergency_brake'
    elif obs_gap < self.dist_brake:   req_acc, self.state = self.acc_brake, 'brake'
else:
    if ttc < self.ttc_emergency:      req_acc, self.state = self.acc_emergency, '
    emergency_brake'
    elif ttc < self.ttc_brake:        req_acc, self.state = self.acc_brake, 'brake'

return req_acc, self.state
```

Listing 3: Reactive Decision Tree

Part II: Abstracted System Dynamics

5 Formal Abstraction Methodology

The formal verification of the AEBS relies on constructing a finite abstraction of the continuous dynamics. Our methodology is directly based on the constructive procedure for finite approximate abstractions of discrete-time stochastic hybrid systems presented by Abate et al. [1]. We extend this paper's concept of a "Markov set-Chain" into an **Interval Markov Decision Process (IMDP)** compatible with modern probabilistic model checkers like PRISM.

5.1 Theoretical Framework vs. Code Implementation

The theoretical abstraction procedure outlines four primary steps, which we have directly mapped into our Python abstraction pipeline:

1. **State Space Partitioning:**
    - *Theory [1]:* The continuous state space is partitioned into a finite set of disjoint regions X_i represented by center points x_i, with a maximum diameter δ.
    - *Implementation:* The `Grid` class defines bounds and resolutions for position, velocity, and acceleration, generating discrete bins and calculating the `cell_radius` (analogous to $\delta/2$).
2. **Kernel Integration:**
    - *Theory [1]:* The nominal transition probabilities between partitions are computed by integrating the continuous stochastic kernel over the target regions.
    - *Implementation:* The `_compute_kernel_accurate` function iterates over potential target cells and evaluates the probability mass inside their boundaries using the exact Gaussian cumulative distribution function (`fast_norm_cdf`).
3. **Error Bounding:**
    - *Theory [1]:* An explicit abstraction error ϵ is computed based on the grid resolution and the Lipschitz continuity constant L of the stochastic kernel.
    - *Implementation:* The error is calculated explicitly in code as `epsilon = cell_volume * L * cell_radius`.
4. **Markov Set-Chain (IMDP) Construction:**
    - *Theory [1]:* Transitions are defined not as scalar probabilities, but as sets or intervals $[p - \epsilon, p + \epsilon]$ that bound the true continuous behavior.
    - *Implementation:* The system constructs PRISM-compatible IMDP transitions, clamping the lower and upper bounds respectively: `p_min = max(1e-6, prob - epsilon)` and `p_max = min(1.0, prob + epsilon)`. The strictly positive floor (10^{-6}) ensures syntactic compatibility with PRISM's IMDP engine while maintaining mathematical validity.

5.2 Synchronization Protocol

To faithfully reproduce the sequential nature of the closed-loop simulation within the PRISM model, a specific synchronization protocol is enforced. This prevents race conditions and ensures that the controller acts only on fresh observations.

The modules synchronize via a shared global variable $t \in \{1, 2, 3\}$, which cycles through the system components in a strict order:

```
def _write_turn_module(self, f):
    f.write("module Turn\n")
    f.write("    t : [1..3] init 1;\n")
    f.write("    [time_step] (t=1) -> (t'=2);\n") # Plant acts
    f.write("    [perceive]  (t=2) -> (t'=3);\n") # Sensors acts
    f.write("    [control]   (t=3) -> (t'=1);\n") # Controller acts
    f.write("endmodule\n\n")
```

Listing 4: PRISM Module Synchronization (from `prism_generator.py`)

6 Computational Challenges and Optimizations

While the theoretical abstraction provides a sound mathematical framework, executing the generated IMDP in the PRISM model checker introduced severe computational and memory bottlenecks. The primary verification environment was constrained to a local machine with a 12GB RAM limit. To successfully verify the continuous physical model within these constraints, we introduced critical algorithmic optimizations.

6.1 Asymmetric Grid Coarsening

The state space is defined by the domains of relative distance, closing speed, and acceleration. To capture the extreme physics of an imminent collision without "tunneling" (probability mass leaking out of the physical simulation bounds), the acceleration limits required widening to $[-15, 10]\, m/s^2$. Initially, this ballooned the system to over 623,000 states, resulting in JVM `OutOfMemoryError` failures during explicit model checking.

To resolve this, we employed asymmetric coarsening: the spatial and velocity resolutions were kept fine, but the acceleration resolution was doubled from 0.25 to 0.5. This safely reduced the state space to approximately 300,000 states without sacrificing critical spatial precision, satisfying the 12GB memory limit.

6.2 Top-k Transition Pruning

Mapping continuous Gaussian distributions to a discrete grid creates an overwhelmingly dense transition matrix. Mathematically, every source state has a non-zero probability of transitioning to millions of target states due to the infinite tails of the normal distribution. To prevent a transition explosion in PRISM, we implemented a Top-k pruning strategy within the integration step.

The algorithm sorts the calculated target candidates by probability mass and retains only the top $k = 3$ most likely target states per transition. The discarded probability mass is safely absorbed into the generalized uncertainty bounds (ϵ), ensuring the model remains formally sound while drastically reducing the model checker's branching factor.

Part III: Formal Verification Results

7 Verification Matrix

The generated IMDP was verified using the PRISM model checker's explicit engine. Because the abstraction utilizes intervals, queries are resolved by an *Adversary*.

- P_{max} **(Worst-Case):** A pessimistic adversary maximizes the probability of collision, revealing the absolute upper bound of systemic risk.
- P_{min} **(Best-Case):** An optimistic adversary minimizes the probability, revealing the baseline risk imposed purely by physical momentum.

Table 2: Formally Bounded Crash Probabilities by Scenario

Scenario	**Initial State (Gap, V)**	P_{min} **(Crash)**	P_{max} **(Crash)**	**Safety Status**
Nominal Cruising	40m, 15m/s	0.0	0.0	Provably Safe
Warning Threshold	25m, 15m/s	≈ 0.0 (6.5×10^{-29})	0.0005%	Microscopic Risk
Critical Braking	15m, 15m/s	0.0001%	27.16%	Vulnerable
Imminent Collision	10m, 20m/s	8.20%	78.70%	Unrecoverable
Post Collision	0m, 0m/s	1.0	1.0	Baseline

The results demonstrate a mathematically rigorous degradation of safety. In the *Nominal Cruising* scenario, the system is provably safe even against a worst-case adversary. In the *Critical Braking* scenario, the system exhibits a 27.16% vulnerability window where an adverse accumulation of sensor noise and physical limits prevents a safe stop.

7.1 Structural Integrity of the Abstraction

To mathematically guarantee that the abstraction does not conflate crash events with safe boundary exits, we verified the structural isolation of the crash state. Model checking the property `Pmax=? [ F "safe_sink" ]` from the initial crash state (`s=0`) yielded a probability of exactly 0.0. This proves that the crash state is strictly absorbing, and probability mass cannot leak into the safe sink post-collision.

8 Future Verification Work

To further validate the computational optimizations employed for this study, future work will involve executing the entire abstraction and verification pipeline on a high-memory compute cluster. This will enable model checking with a much finer grid resolution and a significantly higher transition retention rate (e.g., $k = 10$). The bounds from the high-fidelity model will be compared against the memory-optimized local model to empirically quantify the precision lost due to coarsening and pruning, further substantiating the robustness of the Top-k methodology.

References

[1] A. Abate, A. D'Innocenzo and M. D. Di Benedetto, "Approximate abstractions of stochastic hybrid systems", *IEEE Transactions on Automatic Control*, vol. 56, no. 11, pp. 2688–2694, 2011. DOI: 10.1109/TAC.2011.2160595.